

TP3 – Python

Pr. Houda BENHAR

Exercice 1.

Techniquement, Python utilise des pointeurs (comme dans le langage de programmation C) vers les mêmes objets; si vous faites `y = x`, `x` et `y` pointent vers la même liste. Donc, si on modifie la liste `x`, la liste `y` est modifiée de la même manière. Pour éviter ce problème, il va falloir créer une copie explicite de la liste initiale en utilisant les slices ou bien les fonctions `list()` ou `copy()`.

Exercice 2.

```
nombre = 50
liste_nombres = list(range(1, nombre+1))
diviseurs = []
```

```
for i in liste_nombres:
    if nombre % i == 0:
        diviseurs.append(i)
```

Exercice 3.

```
list_ =[17, 38, 10, 25, 72]
list_cp = list_[:]
list_cp.sort()
print(list_cp)
list_cp.append(12)
print(list_cp)
list_cp.reverse()
print(list_cp)
print(list_cp.index(17))
list_cp.remove(38)
print(list_cp)
print(list_cp[1:3])
print(list_cp[:2])
print(list_cp[2:])
print(list_cp[:])
print(liste[:len(liste) - 1])
```

Exercice 4.

```
ma_liste = [1, 2, 3]
nouv_list = [4, 5, 6]
ma_liste.extend(nouv_list)
print(ma_liste)
```

Exercice 5.

Méthode 1:

```
nombres = [1, 2, 2, 3, 4, 5, 5, 5, 6, 7, 7, 8, 9, 10]
resultat = []
for n in nombres:
    if n not in resultat:
        resultat.append(n)
print(resultat)
```

Méthode 2:

```
nombres = [1, 2, 2, 3, 4, 5, 5, 5, 6, 7, 7, 8, 9, 10]
for num in nombres:
    while(nombres.count(num) > 1):
        nombres.remove(num)
print(nombres)
```

Exercice 6.

```
def find(tup,e):
    for i,t in enumerate(tup):
        if t == e:
            return i
    return -1
my_tuple = ("houda", 5, [0,5])
print(find(my_tuple, [0,5]))
```

Exercice 7.

```
def max_min(tup):
    return max(tup), min(tup))
```

```
my_tuple = (0,6,5)
print(max_min(my_tuple))
```

Exercice 8.

```
def cumul_valeurs(liste_depart):
    liste_finale = []
    resultat = 0
    for nombre in liste_depart:
        resultat = nombre + resultat
        liste_finale.append(resultat)
    return liste_finale
liste_depart = [2, 5, 7, 3, 9, -14, 10, 13, -18, 9]
print(cumul_valeurs(liste_depart))
```

Ou

```
def cumul_valeurs(liste_depart):
    liste_finale = []
    resultat = 0
    for i in range(len(liste_depart)):
        liste_finale.append(sum(liste_depart[:i+1]))
    return liste_finale
liste_depart = [2, 5, 7, 3, 9, -14, 10, 13, -18, 9]
print(cumul_valeurs(liste_depart))
```

Exercice 9.

```
def moyenne_eleves(notes_eleves):
    moyennes = {}
    for eleve, notes in notes_eleves.items():
        moyenne = sum(notes) / len(notes)
        moyennes[eleve] = moyenne
    return moyennes
```

Exemple d'utilisation de la fonction

```
notes_eleves = {
    "Asmaa": [15, 16],
    "Mohamed": [12, 13, 11],
    "Fatima": [17, 18, 16]
}
```

```
resultat = moyenne_eleves(notes_eleves)
print(resultat)
```

Exercice 10.

```
def regroupe_par_auteur(livres):
    auteurs = {}
    for livre in livres:
        if livre["auteur"] not in auteurs:
            auteurs[livre["auteur"]] = []
        auteurs[livre["auteur"]].append(livre["titre"])
    return auteurs
```